1. What is an exception?

Answer:

An exception is an **unexpected or unwanted event** that occurs during the execution of a program, which disrupts the normal flow of instructions. In Java, an exception is represented as an object that describes the error condition.

Example: Dividing a number by zero, trying to access an array index that doesn't exist, or opening a file that is not found.

---

2. List some of the most common types of exceptions that might occur in Java. Give Example.

Answer:

Exception Type	When it occurs	Example
`ArithmeticException`	Division by zero	`int result = 10 / 0;`
`NullPointerException`	Accessing a method/field on a null reference	`String str = null; str.length();`
`ArrayIndexOutOfBoundsException`	Accessing invalid array index	`int[] arr = new int[5]; arr[10] = 100;`
`NumberFormatException`	Converting invalid string to number	`int num = Integer.parseInt("abc");`
`FileNotFoundException`	Opening a file that doesn't exist	`new FileInputStream("missing.txt");`
`IOException`	Input/output operation fails	Reading from a closed stream

Exception Type	When it occurs	Example
`ClassNotFoundException`	Class not found at runtime	`Class.forName("NonExistentClass");`

Code Examples:

java

```
public class ExceptionExamples {
    public static void main(String[] args) {
        // 1. ArithmeticException
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero!");
        }

        // 2. NullPointerException
        try {
            String str = null;
            System.out.println(str.length());
        } catch (NullPointerException e) {
            System.out.println("Null reference accessed!");
        }

        // 3. ArrayIndexOutOfBoundsException
        try {
            int[] arr = new int[3];
            arr[5] = 100;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index out of bounds!");
        }

        // 4. NumberFormatException
        try {
            int num = Integer.parseInt("hello");
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format!");
        }
```

```
    }
}
```

---

3. How many catch blocks can we use with one try block?

Answer:

We can use **multiple catch blocks** with a single try block. There is **no fixed limit** on the number of catch blocks. However, the order must be from **most specific to most general** (i.e., child exceptions before parent exceptions).

Example:

java

```
public class MultipleCatchExample {
    public static void main(String[] args) {
        try {
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[5]); // ArrayIndexOutOfBoundsException
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Specific: Array index issue");
        }
        catch (ArithmeticException e) {
            System.out.println("Specific: Math issue");
        }
        catch (Exception e) {
            System.out.println("General: Any other exception");
        }
    }
}
```

Rules:

- You can have **zero or more** catch blocks (but at least one if no finally block)
- Order matters: child → parent (specific → general)
- Only **one** catch block executes (the first matching exception type)

---

4. What is finally block? When and how is it used? Give a suitable example.

Answer:

The `finally` block is a block of code that **always executes** regardless of whether an exception occurs or not. It is used for **cleanup operations** like closing files, database connections, or releasing resources.

When it executes:

- After try-catch when NO exception occurs
- After catch block when exception occurs
- Even if there is a `return` statement in try or catch
- Even if an exception is thrown but not caught

Example:

java

```
import java.io.*;

public class FinallyExample {
    public static void main(String[] args) {
        FileInputStream file = null;

        try {
            file = new FileInputStream("sample.txt");
            int data = file.read();
            System.out.println("File data: " + data);
        }
        catch (FileNotFoundException e) {
            System.out.println("File not found!");
        }
        catch (IOException e) {
            System.out.println("Error reading file!");
        }
        finally {
```

```
            // Cleanup code - always executes
            try {
                if (file != null) {
                    file.close();
                    System.out.println("File closed successfully!");
                }
            }
            catch (IOException e) {
                System.out.println("Error closing file!");
            }
            System.out.println("Finally block always runs!");
        }
    }
}
```

Output (if file exists):

text

```
File data: 72
File closed successfully!
Finally block always runs!
```

Output (if file doesn't exist):

text

```
File not found!
Finally block always runs!
```

---

5. Write code to demonstrate how throw keyword works?

Answer:

The `throw` keyword is used to **explicitly throw an exception** (either built-in or custom) from any method or block.

Example 1: Throwing built-in exception

java

```
public class ThrowDemo {

    public static void validateAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Age must be 18 or above. Given: " + age);
        }
        System.out.println("Valid age: " + age);
    }

    public static void main(String[] args) {
        System.out.println("Checking age 20:");
        validateAge(20);  // Valid

        System.out.println("\nChecking age 15:");
        try {
            validateAge(15);  // This will throw exception
        } catch (IllegalArgumentException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }

        System.out.println("\nProgram continues normally...");
    }
}
```

Output:

text

```
Checking age 20:
Valid age: 20

Checking age 15:
Exception caught: Age must be 18 or above. Given: 15

Program continues normally...
```

Example 2: Throwing custom exception

java

```
// Custom exception class
class LowBalanceException extends Exception {
```

```
    public LowBalanceException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance = 500;

    public void withdraw(double amount) throws LowBalanceException {
        if (amount > balance) {
            throw new LowBalanceException("Insufficient balance! Available: $" + balance);
        }
        balance -= amount;
        System.out.println("Withdrawn: $" + amount + ", Remaining: $" + balance);
    }
}

public class ThrowCustomDemo {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        try {
            account.withdraw(100);   // Valid - withdraws
            account.withdraw(1000);  // Invalid - throws exception
        } catch (LowBalanceException e) {
            System.out.println("Transaction failed: " + e.getMessage());
        }
    }
}
```

Output:

text

```
Withdrawn: $100.0, Remaining: $400.0
Transaction failed: Insufficient balance! Available: $400.0
```

Key Points about `throw`:

Feature	Description
Purpose	Manually throw an exception
Location	Inside method body
Usage	`throw new ExceptionType("message");`
Followed by	An instance of Throwable class
Checked exception	Must be declared with `throws` in method signature
